

A Tree-Based Syntactic Complexity Analyzer for English Language Learners

CSC111 Project 2 Final Report

Tugra Canbaz Elif Cakici Alsade Brianna Daley

April 1, 2026

Introduction

Research Question: How can we model English sentences using parse trees and design custom tree-based algorithms to measure syntactic complexity, in order to provide useful, structured feedback to English language learners?

When learning a new language, tracking genuine progress is one of the hardest challenges a learner faces. Standardized tests and vocabulary quizzes offer one dimension of measurement, but they do not capture how sophisticated a learner’s sentence structures have become. Vocabulary size alone is a poor proxy for fluency: the way a learner arranges clauses, embeds subordinate structures, and builds hierarchical phrases reveals far more about their actual development than any word count. One of the most well-established indicators of linguistic growth is *syntactic complexity*, the structural richness of the sentences a learner produces [2].

In linguistics and computational linguistics, sentence structure is most naturally represented using *parse trees*. A parse tree decomposes a sentence into its grammatical constituents, noun phrases (NP), verb phrases (VP), subordinate clauses (SBAR), and so on, and arranges them in a hierarchy that reflects English’s recursive syntactic structure. The depth of this tree, the number of subordinate clauses it contains, and the average branching at each node are all measurable signals of structural complexity. Prior research confirms that tree-based features correlate meaningfully with CEFR proficiency levels, with more advanced learners consistently producing sentences of greater hierarchical depth and clause density [6].

Our project builds a **Tree-Based Sentence Complexity Analyzer** that evaluates English sentences using syntactic parse trees generated through the NLTK library [1, 4]. The system extracts three core structural metrics, tree depth, clause count, and branching factor, combines them into a weighted complexity score, maps that score to an approximate CEFR level (A1 through C2), and returns personalized, metric-tied feedback along with upgraded sentence suggestions. When run on learner corpus data, the system also generates bar charts and a summary table comparing average complexity metrics across proficiency levels, allowing visual validation that syntactic complexity increases monotonically with CEFR level.

Datasets

EFCAMDAT Cleaned Sub-Corpus

The Education First Cambridge Open Language Database (EFCAMDAT) [3] is a large-scale learner corpus. We use the publicly distributed *cleaned sub-corpus* prepared by Shatz (2020), which covers levels 1–15 for learners from eleven nationalities: Brazilian, Chinese, Taiwanese, Russian, Saudi Arabian, Mexican, German, Italian, French, Japanese, and Turkish. The cleaning pipeline removed common markup tags, ultra-short texts (under 20 words), texts with substantial non-English content, near-duplicate texts, and texts with outlier word counts. Texts were additionally split by prompt type; we used the file corresponding to the main (first) prompt: `Final database (main prompts).xlsx`.

We selected 100 rows from each cefr level to create a smaller database, and converted it to a csv file called `small_data_cleaned.csv`. Our program reads this file using the `pandas` library [8] via `pd.read_csv()`. The only column used is the text column containing learner-written responses (the raw sentence strings). Proficiency level labels, where present, allow grouping for aggregate analysis. None of the other columns were used.

Dataset Download Instructions

The `small_data_cleaned.csv` file was submitted to MarkUs alongside this report. Place this file in the same directory as `main.py` before running the program.

Computational Overview

Data Representation with ParseTree Class

The central data structure of this project is the `ParseTree` class, defined in `parse_tree.py`. Each `ParseTree` node stores a string `label`, either a grammatical tag such as 'NP', 'VP', or 'SBAR' for internal nodes, or a word token string for leaf nodes, and a private list `_subtrees` of child `ParseTree` objects. Two representation invariants hold throughout the program: `label` is never empty, and a node is a leaf if and only if its subtree list is empty.

Trees are the appropriate data structure for this domain because English syntax is inherently recursive and hierarchical [2]. A flat list of words cannot represent the structural relationship between a subject noun phrase and its predicate verb phrase, let alone the nesting of a relative clause inside a prepositional phrase inside a verb phrase. The `ParseTree` class supports all operations needed for metric computation: `get_subtrees()` returns the list of children, `is_leaf()` identifies terminal nodes, `add_subtree()` builds the tree during parsing, and `find_subtree_by_label()` supports targeted structural queries.

Building Parse Trees from Sentences

Raw sentences are converted into `ParseTree` instances through a two-stage pipeline.

Stage 1: NLTK parsing. The function `sentence_to_parsetree(sentence)` in `parse_tree.py` tokenizes the input string using NLTK's `word_tokenize()` [1], assigns part-of-speech tags using `nltk.pos_tag()`, and then parses the tagged token sequence using a `RegexParser` with a hand-written context-free grammar. The grammar (the module-level constant `_GRAMMAR`) defines phrase patterns for NP, PP, ADJP, VP, SBAR, and S using regular expressions over POS tag sequences. The `RegexParser.parse()` method produces an `nltk.Tree` object representing the constituent structure of the sentence.

Stage 2: Conversion to custom tree. The function `convert_to_parsetree(nltk_tree)` in `parse_tree.py` recursively converts this NLTK-native tree into our custom `ParseTree`. Each `nltk.Tree` node becomes a `ParseTree` with the same grammatical label; each `(word, pos_tag)` leaf tuple becomes a `ParseTree` leaf whose label is the word string (the POS tag is discarded). This separation of concerns keeps the NLTK dependency contained to the parsing stage, while all downstream metric computations operate exclusively on our own `ParseTree` class.

Dataset sentences are processed through the same pipeline inside `data_analysis.py`. The function `process_sentence()` first calls `clean_sentence()` to normalize whitespace using a regular expression (`re.sub`), then tokenizes and POS-tags using NLTK, parses with the `RegexParser`, and calls `convert_to_parsetree()` to return a custom `ParseTree`. The `if __name__ == "__main__"` block reads `small_data_cleaned.csv` via `pd.read_csv()` and computes averages per CEFR level.

Metric Computations

All syntactic metrics are implemented as recursive functions in `parse_tree.py`, operating on `ParseTree` objects. Each function traverses the tree in $O(n)$ time, where n is the number of nodes.

`get_depth(tree)` returns the length of the longest root-to-leaf path. A single leaf has depth 0; an internal node's depth is one plus the maximum depth of its children, computed via a recursive generator expression over `get_subtrees()`.

`count_clauses(tree)` counts all nodes whose label is 'S' or 'SBAR', including the root if it qualifies. Clause count is a direct measure of how many distinct propositional units a sentence expresses; sentences with more embedded clauses score higher [6].

`branching_factor(tree)` computes the mean number of children across all internal (non-leaf) nodes, using a nested helper `_get_values()` that collects child counts into a list before averaging. A high branching factor indicates that phrases are expanded with many modifiers and arguments.

`count_nodes(tree)` and `count_leaves(tree)` are auxiliary metrics counting total nodes and word-token leaves respectively, used for diagnostic inspection.

These three primary metrics are combined in `compute_score(depth, clause_count, branch)` in `feedback.py` using a weighted formula:

$$\text{score} = 0.4 \times \text{depth} + 0.3 \times \text{clause_count} + 0.3 \times \text{branch}$$

The result is rounded to two decimal places. The score is then mapped to a CEFR level string by `classify_level(score)`, which applies threshold comparisons: $\text{score} < 2$ maps to A1, $\text{score} < 3$ to A2, $\text{score} < 4$ to B1, $\text{score} < 5$ to B2, and $\text{score} \geq 5$ to C1/C2 Advanced.

Feedback Generation

The function `generate_feedback(sentence, depth, clause_count, branch)` in `feedback.py` produces personalized, metric-tied feedback by comparing each metric against threshold ranges. A depth of 2 or below triggers a note about shallow structure and a suggestion to add a descriptive phrase; a clause count of 1 triggers three alternative rewrites (adding a reason, a contrast, and a conditional); low branching triggers suggestions to add adjectives or prepositional phrases. The function returns a `tuple` of two lists: `feedback` (observations about current structure) and `suggestions` (concrete rewrites anchored to the input sentence). The companion function `suggest_upgrade(sentence)` additionally produces four upgraded versions of the sentence by appending subordinate clauses of varying types.

The full pipeline is orchestrated by `analyze_sentence(sentence)` in `feedback.py`, which also calls `_download_nltk_resources()` to ensure the required NLTK data packages (`punkt_tab` and `averaged_perceptron_tagger_eng`) are present before any parsing occurs.

Visualizations and Results Display

The `visualizations.py` module generates four `matplotlib` [7] charts and a formatted console table summarizing complexity metrics grouped by CEFR level (A1 through C2):

- **Graph 1** (`graph1_complexity_score.png`): A bar chart of average complexity score per level, with value labels placed above each bar using `ax.text()`.
- **Graph 2** (`graph2_depth.png`): A bar chart of average parse tree depth per level.
- **Graph 3** (`graph3_clause_count.png`): A bar chart of average clause count per level.
- **Graph 4** (`graph4_distribution.png`): An overlapping histogram (`ax.hist()`) showing the distribution of complexity scores within each CEFR level, revealing both central tendency and variance at each proficiency stage.
- **Results Table**: Printed to the console by `print_results_table()`, showing average depth, clause count, and complexity score for all six levels in a formatted tabular layout using aligned f-strings.

Each chart is produced using `plt.subplots()`, styled with a shared six-color palette, and saved as a PNG file at 150 dpi via `plt.savefig()` before being displayed with `plt.show()`. The `matplotlib.pyplot` module's bar, histogram, axis-labeling, and grid functions are central to this module.

Instructions for Obtaining Datasets and Running the Program

Requirements

Install all required Python libraries using the provided `requirements.txt`:

```
nltk
matplotlib
python-ta
pandas
```

Obtaining the Datasets

The dataset file has been uploaded to MarkUs as part of this submission. After downloading and extracting, place `small_data_cleaned.csv` in the same directory as `main.py`

Running the Program

Run the entry point:

```
python main.py
```

The program proceeds through the following stages automatically:

1. **NLTK setup.** Required NLTK data packages are downloaded silently on first run.
2. **Test cases.** Four built-in sentences spanning A1 to C2 difficulty are analyzed. For each sentence, the console displays a formatted block containing: the input sentence, its complexity score, estimated CEFR level, individual metric values (depth, clause count, branching factor), a bulleted feedback section, specific rewrite suggestions, and two upgraded sentence versions. For example, the sentence *“I like school.”* will score around 1.4 and be classified as A1 – Beginner, while *“The results, which were published last year, suggest a strong correlation.”* will score higher and reach B2 or above.
3. **Interactive mode.** The program prompts: `Enter a sentence (or press Enter to skip):`. The user may type any English sentence and press Enter to receive a full analysis, or press Enter immediately to skip this step.
4. **Results table.** A formatted CEFR summary table is printed to the console, showing average depth, clause count, and complexity score for A1 through C2.
5. **Visualizations.** Four matplotlib chart windows open in sequence. Each window must be closed before the next appears. All four charts are also saved as PNG files in the working directory: `graph1_complexity_score.png`, `graph2_depth.png`, `graph3_clause_count.png`, and `graph4_distribution.png`.

Changes from the Project Proposal

Several aspects of the project evolved between the proposal and the final submission.

First, the EFCAMDAT corpus access, which was pending institutional approval at proposal time, was resolved by using the publicly distributed cleaned sub-corpus (Shatz, 2020) [3] rather than the full original database. This made the dataset immediately accessible and the cleaned version is better-suited for our analysis since noise, outliers, and near-duplicates have already been removed.

Second, the complexity scoring formula was simplified. The proposal described a Wolfram-inspired approach [5] incorporating graph diameter and mean node distance in addition to tree depth. After implementation, these graph-theoretic measures added pipeline complexity without meaningfully improving the

correlation between score and CEFR level in manual testing. We simplified to the three-feature weighted formula (depth, clause count, branching factor), which is more interpretable and more directly tied to the feedback messages.

Third, the feedback system was significantly expanded. Rather than simply printing a score and level, the final program generates sentence-specific suggestions anchored to the user’s own input text and produces upgraded rewrite examples via `suggest_upgrade()`. This makes the tool genuinely instructional rather than merely diagnostic.

Fourth, an interactive mode was added to `main.py` via `run_interactive()`, allowing any user to analyze their own sentence at runtime. This was not part of the original proposal.

Fifth, a dedicated data pipeline module (`data_analysis.py`) was added to handle the EFCAMDAT dataset. The proposal anticipated processing this corpus but did not specify a separate module for it.

Discussion

The results of our program are consistent with the core premise of the project: syntactic complexity, as measured by parse tree depth, clause count, and branching factor, increases meaningfully across CEFR proficiency levels. The bar charts generated by `visualizations.py` show a clear monotonic trend in all three metrics from A1 to C2, and the composite score cleanly separates the six levels without overlap in their average values. This is consistent with findings in the second language acquisition literature, which documents that structural elaboration; more clauses, deeper embedding, greater phrase modification, is a reliable marker of proficiency growth [6, 2].

However, several important limitations should be acknowledged.

The most significant limitation is the `RegexpParser`. Unlike a statistical or neural constituency parser (e.g., the Berkeley Neural Parser or spaCy’s dependency parser), `RegexpParser` uses deterministic, hand-written rules. It misses many syntactically valid constructions that do not match its patterns, and its parse quality degrades on longer or more unusual sentences. Our tree depth and clause count values are, therefore, approximate lower bounds on true syntactic complexity. For a production-grade tool, a more powerful parser would be essential.

A second limitation is that the scoring weights (0.4, 0.3, 0.3) and the CEFR level thresholds in `classify_level()` were chosen by inspection rather than fitted on labeled data. The `_CEFR_DATA` values are computed by running `compute_averages()` in `data_analysis.py` over the EFCAMDAT cleaned sub-corpus, producing real per-level averages for depth, clause count, branching factor, and complexity score. A natural next step is to run `data_analysis.py` over the full EFCAMDAT corpus, compute real per level averages, and use those to calibrate both the thresholds and the chart data.

A third limitation is that our approach is purely structural and ignores lexical content entirely. A sentence with sophisticated vocabulary but simple syntax would receive a low score, which does not reflect its true communicative complexity. Incorporating lexical diversity measures (type-token ratio) or average word frequency would produce a more complete picture.

Despite these limitations, the project successfully demonstrates that custom tree data structures and recursive algorithms can serve as the backbone of a practically useful NLP tool, connecting core CSC111 concepts (tree traversal, recursive computation, abstract data types) directly to a real-world application in language education technology. Future work could integrate a pre-trained parser, fit scoring weights on labeled corpus data, and extend the feedback system to cover lexical and discourse-level features.

References

- [1] Bird, S., Klein, E., and Loper, E. *Natural Language Processing with Python*. O'Reilly Media, 2009. Available: <https://www.nltk.org/>
- [2] Center for Advanced Research on Language Acquisition (CARLA), “Overview of Complexity of Learner Language,” University of Minnesota, n.d. Available: <https://carla.umn.edu/learnerlanguage/complexity.html>
- [3] Shatz, I. “Refining and Modifying the EFCAMDAT: Lessons from Creating a New Corpus from an Existing Large-Scale English Learner Language Database.” *International Journal of Learner Corpus Research*, vol. 6, no. 2, pp. 220–236, 2020. Available: <https://ef-lab.mml1.cam.ac.uk/EFCAMDAT.html>
- [4] Natural Language Toolkit (NLTK) Project. *NLTK 3.x Documentation*, 2024. Available: <https://nltk.readthedocs.io/en/latest/>
- [5] Wolfram Research, “SentenceComplexityMeasures,” *Wolfram Function Repository*, 2021. Available: <https://resources.wolframcloud.com/FunctionRepository/resources/SentenceComplexityMeasures/>
- [6] Zhang, X. and Lu, X. “Aligning Linguistic Complexity with the Difficulty of English Texts for L2 Learners Based on CEFR Levels.” *Studies in Second Language Acquisition*, vol. 47, no. 5, pp. 1407–1434, 2025. Available: <https://doi.org/10.1017/S0272263125101125>
- [7] Hunter, J. D. “Matplotlib: A 2D Graphics Environment.” *Computing in Science and Engineering*, vol. 9, no. 3, pp. 90–95, 2007. Available: <https://matplotlib.org/>
- [8] McKinney, W. “Data Structures for Statistical Computing in Python.” *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010. Available: <https://pandas.pydata.org/>